

PRACTICAL SHEET – 7 (ARYAN KUSHWAHA - 245703428)

Q1: Implement a standard Trie

```
#include <iostream>
#include <string>
#include <vector>

using namespace std;

class TrieNode {
public:
    TrieNode* children[26];
    bool isTerminal;

    TrieNode() {
        isTerminal = false;
        for (int i = 0; i < 26; i++) {
            children[i] = nullptr;
        }
    }
};

class Trie {
private:
    TrieNode* root;

public:
    Trie() {
        root = new TrieNode();
    }

    void insert(const string& word) {
        TrieNode* curr = root;
        for (char c : word) {
            int index = c - 'a';
            if (!curr->children[index]) {
                curr->children[index] = new TrieNode();
            }
            curr = curr->children[index];
        }
        curr->isTerminal = true;
    }

    bool search(const string& word) {
        TrieNode* curr = root;
        for (char c : word) {
            int index = c - 'a';
            if (!curr->children[index]) {
                return false;
            }
            curr = curr->children[index];
        }
        return curr->isTerminal;
    }
};
```

```

56  int main() {
57      Trie trie;
58      int n, q;
59
60      cout << "Enter the number of words to insert: ";
61      cin >> n;
62      cout << "Enter " << n << " words:\n";
63      for (int i = 0; i < n; i++) {
64          string word;
65          cin >> word;
66          trie.insert(word);
67      }
68
69      cout << "Enter the number of search queries: ";
70      cin >> q;
71      cout << "Enter " << q << " queries:\n";
72      for (int i = 0; i < q; i++) {
73          string word;
74          cin >> word;
75          if (trie.search(word)) {
76              cout << word << " found in Trie.\n";
77          } else {
78              cout << word << " not found in Trie.\n";
79          }
80      }
81
82      return 0;
83  }

```

Output:

```

Enter the number of words to insert: 4
Enter 4 words:
sheet
sheets
short
shaft
Enter the number of search queries: 5
Enter 5 queries:
sheet
sheet found in Trie.
sheets
sheets found in Trie.
shafts
shafts not found in Trie.
shell
shell not found in Trie.
shorts
shorts not found in Trie.

```

Q2: Implement a complex Trie.

```
3  #include <iostream>
4  #include <string>
5  #include <unordered_map>
6
7  using namespace std;
8
9  class ComplexNode {
10 public:
11     unordered_map<char, ComplexNode*> children;
12     int wordsEndingHere = 0;
13     int prefixesThroughHere = 0;
14 };
15
16 class ComplexTrie {
17 private:
18     ComplexNode* root;
19
20 public:
21     ComplexTrie() {
22         root = new ComplexNode();
23     }
24
25     void insert(const string& word) {
26         ComplexNode* curr = root;
27         for (char c : word) {
28             if (curr->children.find(c) == curr->children.end()) {
29                 curr->children[c] = new ComplexNode();
30             }
31             curr = curr->children[c];
32             curr->prefixesThroughHere++;
33         }
34         curr->wordsEndingHere++;
35     }
36
37     int countWordsEqualTo(const string& word) {
38         ComplexNode* curr = root;
39         for (char c : word) {
40             if (curr->children.find(c) == curr->children.end()) return 0;
41             curr = curr->children[c];
42         }
43         return curr->wordsEndingHere;
44     }
45
46     void erase(const string& word) {
47         if (countWordsEqualTo(word) == 0) return;
48
49         ComplexNode* curr = root;
50         for (char c : word) {
51             curr = curr->children[c];
52             curr->prefixesThroughHere--;
53         }
54         curr->wordsEndingHere--;
55     }
56 };
```

```

58  int main() {
59      ComplexTrie trie;
60      int choice;
61      string word;
62
63      cout << "1. Insert\n2. Count Words\n3. Erase\n4. Exit\n";
64      while (true) {
65          cout << "Enter choice: ";
66          cin >> choice;
67
68          if (choice == 1) {
69              cin >> word;
70              trie.insert(word);
71          } else if (choice == 2) {
72              cin >> word;
73              cout << "Count: " << trie.countWordsEqualTo(word) << "\n";
74          } else if (choice == 3) {
75              cin >> word;
76              trie.erase(word);
77          } else {
78              break;
79          }
80      }
81
82      return 0;
83  }

```

Output:

```

1. Insert
2. Count Words
3. Erase
4. Exit
Enter choice: 1
comment
Enter choice: 1
comment
Enter choice: 1
com
Enter choice: 2
comment
Count: 2
Enter choice: 3
comment
Enter choice: 2
comment
Count: 1
Enter choice: 4

```

Q3: Implement a Suffix Trie.

```
3  #include <iostream>
4  #include <string>
5  #include <unordered_map>
6
7  using namespace std;
8
9  class SuffixNode {
10 public:
11     unordered_map<char, SuffixNode*> children;
12 };
13
14 class SuffixTrie {
15 private:
16     SuffixNode* root;
17     char endSymbol;
18
19     void insertSuffix(int index, const string& str) {
20         SuffixNode* curr = root;
21         for (int i = index; i < str.length(); i++) {
22             char c = str[i];
23             if (curr->children.find(c) == curr->children.end()) {
24                 curr->children[c] = new SuffixNode();
25             }
26             curr = curr->children[c];
27         }
28         curr->children[endSymbol] = nullptr;
29     }
30
31 public:
32     SuffixTrie(const string& str) {
33         root = new SuffixNode();
34         endSymbol = '*';
35         populateSuffixTrie(str);
36     }
37
38     void populateSuffixTrie(const string& str) {
39         for (int i = 0; i < str.length(); i++) {
40             insertSuffix(i, str);
41         }
42     }
43
44     bool contains(const string& substring) {
45         SuffixNode* curr = root;
46         for (char c : substring) {
47             if (curr->children.find(c) == curr->children.end()) {
48                 return false;
49             }
50             curr = curr->children[c];
51         }
52         return true;
53     }
54 };
```

```

56  int main() {
57      string text;
58      cout << "Enter the main string to build the Suffix Trie: ";
59      cin >> text;
60
61      SuffixTrie trie(text);
62
63      int q;
64      cout << "Enter number of substring queries: ";
65      cin >> q;
66
67      cout << "Enter queries:\n";
68      for (int i = 0; i < q; i++) {
69          string query;
70          cin >> query;
71          if (trie.contains(query)) {
72              cout << "'" << query << "' is a substring.\n";
73          } else {
74              cout << "'" << query << "' is NOT a substring.\n";
75          }
76      }
77
78      return 0;
79  }

```

Output:

```

Enter the main string to build the Suffix Trie: example
Enter number of substring queries: 3
Enter queries:
ex
'ex' is a substring.
exam
'exam' is a substring.
exams
'exams' is NOT a substring.

```

Q4: Implement bellman ford algorithm.

```
3  #include <iostream>
4
5
6  using namespace std;
7
8  struct Edge {
9      int src, dest, weight;
10 };
11
12 void bellmanFord(int V, int E, int source, const vector<Edge>& edges) {
13     vector<int> dist(V, 1e9);
14     dist[source] = 0;
15
16     for (int i = 1; i <= V - 1; i++) {
17         for (int j = 0; j < E; j++) {
18             int u = edges[j].src;
19             int v = edges[j].dest;
20             int weight = edges[j].weight;
21
22             if (dist[u] != 1e9 && dist[u] + weight < dist[v]) {
23                 dist[v] = dist[u] + weight;
24             }
25         }
26     }
27
28     for (int j = 0; j < E; j++) {
29         int u = edges[j].src;
30         int v = edges[j].dest;
31         int weight = edges[j].weight;
32         if (dist[u] != 1e9 && dist[u] + weight < dist[v]) {
33             cout << "Graph contains a negative weight cycle\n";
34             return;
35         }
36     }
37
38     cout << "Vertex\tDistance from Source\n";
39     for (int i = 0; i < V; i++) {
40         if (dist[i] == 1e9) cout << i << "\tINF\n";
41         else cout << i << "\t" << dist[i] << "\n";
42     }
43 }
```

```

45 int main() {
46     int V, E;
47     cout << "Enter number of vertices and edges: ";
48     cin >> V >> E;
49
50     vector<Edge> edges(E);
51     cout << "Enter " << E << " edges (source destination weight):\n";
52     for (int i = 0; i < E; i++) {
53         cin >> edges[i].src >> edges[i].dest >> edges[i].weight;
54     }
55
56     int source;
57     cout << "Enter the source vertex: ";
58     cin >> source;
59
60     bellmanFord(V, E, source, edges);
61
62     return 0;
63 }

```

Output:

```

Enter number of vertices and edges: 4 4
Enter 4 edges (source destination weight):
0 1 4
0 2 5
2 1 -2
1 3 3
Enter the source vertex: 0
Vertex Distance from Source
0      0
1      3
2      5
3      6

```